

Task 1 – Maze Challenge (OpenCV)

Vihaan Jain

251080026

Information Technology

Drive Link :-

https://drive.google.com/file/d/1IgRz-8RU3esXzBfOMvvGuKHajAquFFBk/view?usp=drive_link

When I first saw the problem statement, I honestly thought this was just going to be a BFS problem. My plan was pretty simple somehow convert the image into a grid, run BFS, get the shortest path and repeat it for all 100 mazes.

After generating the images and opening a few of them, I realised it wasn't that simple. The path finding part was actually easy. The difficult part was figuring out what exactly the image was representing and how to convert it into a proper maze.

The task was to process 100 maze images, find the shortest path from the top left corner to the bottom right corner, ignore the impossible mazes and multiply all valid path lengths together modulo $10^9 + 7$ to get the final password.

At the beginning I thought most of my time would go into BFS. That ended up being completely wrong.

Initial Observations

The first thing I did was zoom into a few generated mazes and just stare at them for a while. At first everything looked straightforward. There were red walls and white paths, so I assumed identifying walls would be easy.

After looking more carefully, I started noticing some weird stuff.

There were random gray pixels in different places. They looked like noise and didn't clearly belong to either walls or paths.

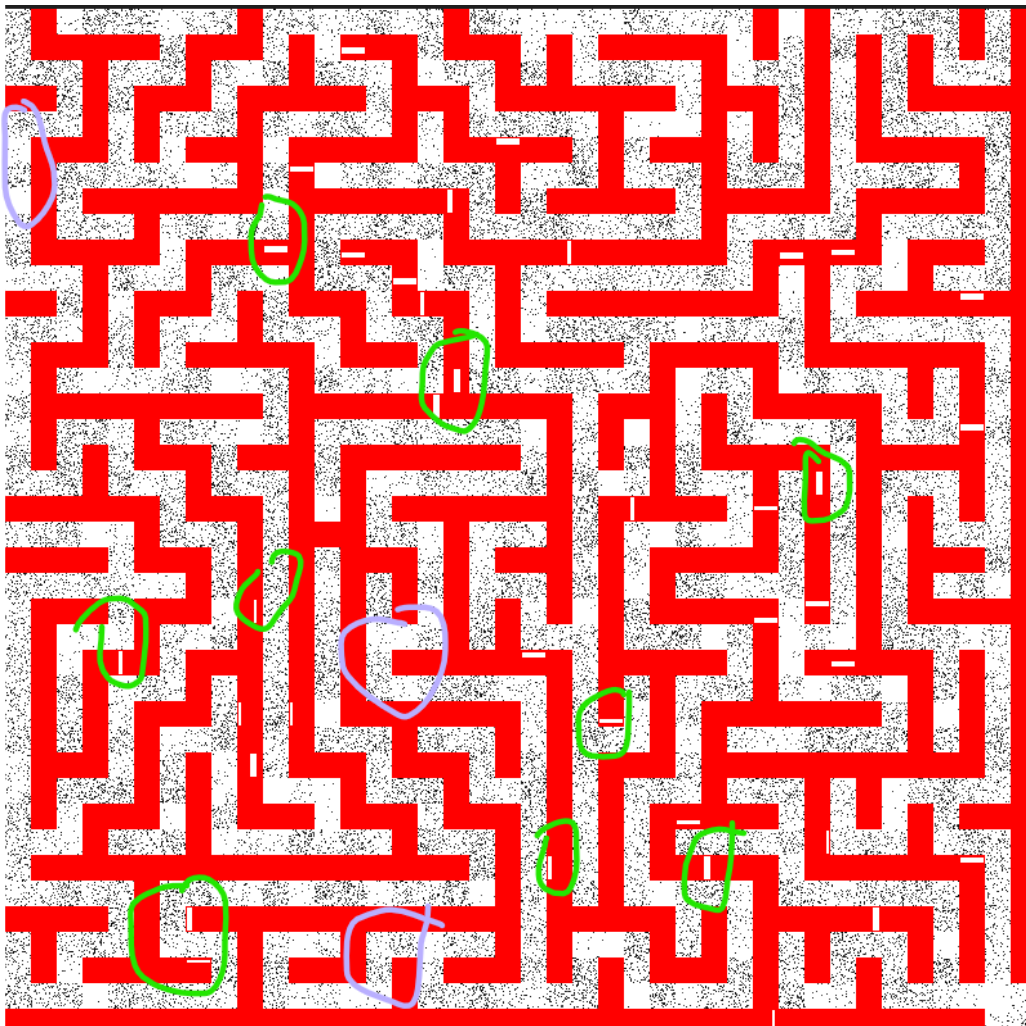
Then I noticed small white cuts inside some of the red walls. At first I treated them as actual openings. Since they looked very similar to normal passages, I assumed they were part of the maze itself.

After checking more images, I started getting suspicious. Some of these white gaps appeared in places where a valid passage didn't even make sense. That was when I realised they were probably there just to confuse people.

I also noticed that cells near walls often contained mixed colours. A path cell could have a few red pixels near the edge and a wall cell could have a few lighter pixels near the boundary. Because of this, checking a single pixel was clearly not going to work.

At that point I realised solving the maze directly at pixel level would probably create more problems than it solved.

Fig : Shows maze having white “cuts” inside walls which causes issues.



Early Attempts

My first idea was direct colour thresholding.

I tried identifying wall pixels using colour values and then converting the image into a binary maze.

It worked sometimes, but the results became inconsistent whenever noise appeared near wall boundaries.

I also tried a few resizing based ideas and played around with different threshold values. Some outputs actually looked correct at first. The paths connected the start and end points and everything looked fine.

Then I started manually checking some solved images.

That's when I noticed something strange. A few paths looked like they were cutting through walls.

For some time I genuinely thought my BFS implementation was broken. I checked neighbour generation, visited states, path reconstruction and pretty much everything related to traversal.

Eventually I realised BFS was doing exactly what it was supposed to do. The actual mistake was happening much earlier during maze extraction.

That was probably the biggest turning point of the project.

Understanding the Maze Structure

After getting stuck for a while, I went back and read the problem statement again.

This time I paid attention to something I had mostly ignored before.

The statement clearly mentioned that the maze is a logical 40×40 grid and each logical cell occupies exactly 20×20 pixels.

Looking back, this was basically the biggest clue in the entire challenge and it was sitting there from the beginning.

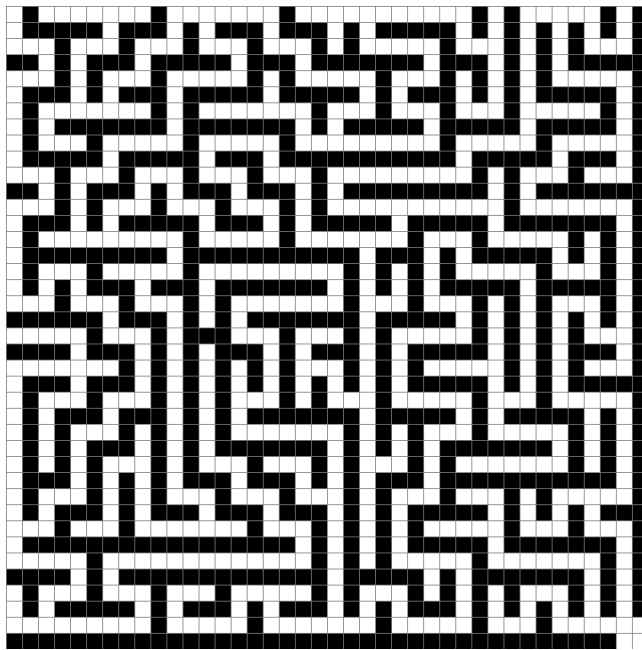
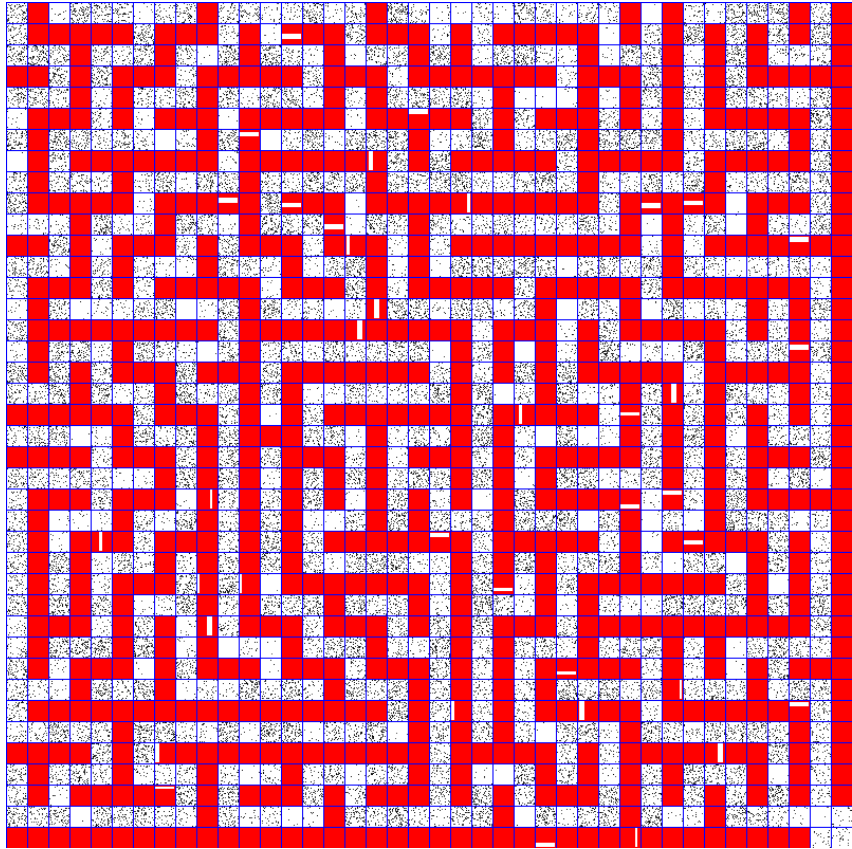
Instead of treating the image as a huge collection of pixels, I started treating it as a collection of logical cells.

Every cell already had a fixed position and fixed dimensions.

The problem became much simpler.

For every 20×20 cell, decide whether it is a wall or a path.

Once I switched to this approach, everything started making a lot more sense.



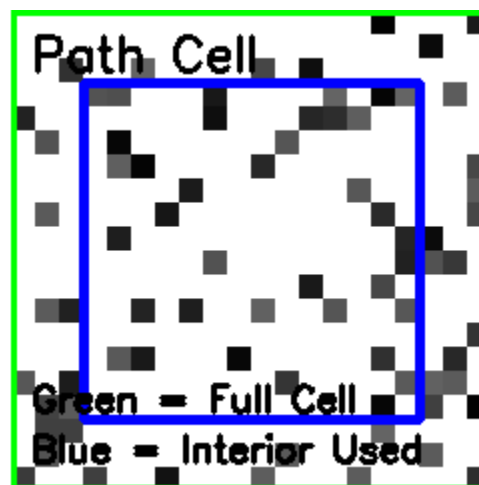
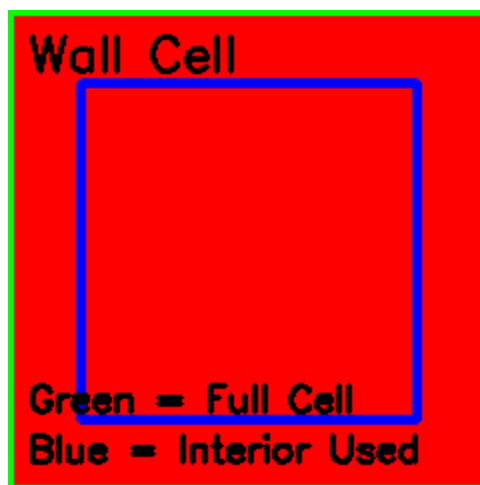
Final Approach

Image Preprocessing

Before extracting the grid, I applied morphological closing using OpenCV. To be honest, I wasn't even sure if this step was necessary when I first added it. I tested the solution both with and without it. The final password stayed the same, but the wall regions looked a little cleaner after preprocessing, so I decided to keep it. The goal wasn't to change the maze itself. I only wanted to reduce the effect of small visual artifacts before classification.

Cell Classification

This ended up being the most important part of the entire solution. In the code, the `iswall()` function decides whether a logical cell represents a wall. Instead of analysing the full 20×20 region, I ignored a small border around the cell and only looked at the interior. I did this because most colour bleeding happened near the edges. The walls were represented using a strong red colour, so the program identifies red wall pixels inside the interior region and counts them. Instead of checking a single pixel, I used a majority vote approach. If more than half of the interior pixels looked like wall pixels, I marked the entire cell as a wall. Otherwise it was treated as an open path. This worked way better than the earlier methods I had tried.



Maze Reconstruction

The `extract()` function goes through all 40×40 logical cells and builds a binary maze representation.

At this stage, the image processing part is basically done.

The maze now exists as a normal grid which can be solved using graph algorithms.

Path Finding

The `bfs()` function is used to find the shortest path.

Since all moves have equal cost, BFS guarantees the shortest path in the grid.

The algorithm starts at the top left cell and explores neighbouring cells until the destination is reached.

Parent information is stored while traversing so the final path can later be reconstructed.

Honestly, I wrote the BFS part in maybe twenty minutes. After spending so much time trying to understand the images, BFS felt like the easy part.

Output Generation

The `draw()` function creates solved maze images.

For every solvable maze, the recovered shortest path is drawn on top of the original image.

For impossible mazes, a separate output image is generated.

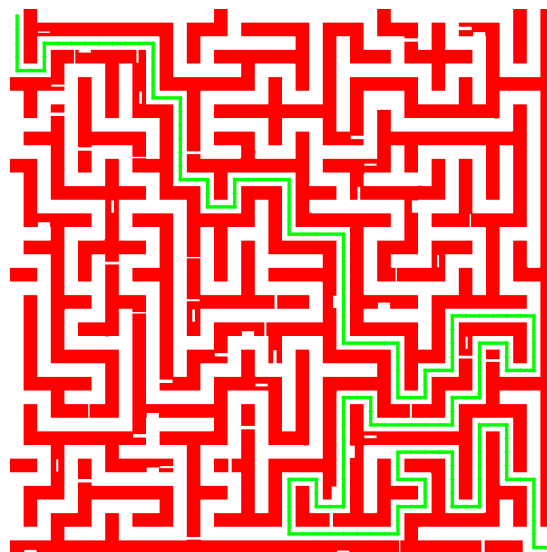
This made it easy to visually verify that the extracted maze was actually correct.

Password Generation

For every solvable maze, the shortest path length is multiplied into the running answer modulo $10^9 + 7$.

Impossible mazes are skipped completely.

After all 100 mazes are processed, the final password is obtained.



Challenges Faced

The thing that confused me the most was definitely the white gaps inside walls.

For a long time I treated them as genuine openings because visually they looked almost identical to actual passages.

I remember checking one of the later mazes, somewhere around maze_073, because the generated path looked suspicious. After tracing it manually I realised the path was effectively travelling through a wall region.

That was the point where I stopped trusting visual appearance alone and started verifying the logical grid itself.

Another challenge was deciding whether to work at pixel level or logical cell level.

I spent quite a bit of time experimenting with pixel based approaches before finally accepting that the logical grid information given in the problem statement was the correct way to approach the problem.

There were also moments where I genuinely thought BFS was broken because some outputs looked completely wrong.

After a lot of debugging I discovered that BFS was working perfectly fine. The real issue was always the maze extraction.

Looking back, most of my time went into understanding the images rather than writing algorithms.

Results

Total Maze Files Processed : 100

Solvable Mazes : 79

Impossible Mazes : 21

Final Password : 493691789

I ran the solver multiple times after making small changes during debugging just to make sure I hadn't broken anything.

Every successful run produced the same password, 493691789, which gave me confidence that the implementation was stable.

The 21 impossible mazes were actually more than I expected. I manually checked a few of them and confirmed that there genuinely was no valid path.